

Fast Fiber Line Extraction for 2D Bivariate Scalar Fields

Felix Raith^{id*}
Leipzig University

Baldwin Nsonga^{id†}
Leipzig University

Gerik Scheuermann^{id‡}
Leipzig University

Christian Heine^{id§}
Leipzig University

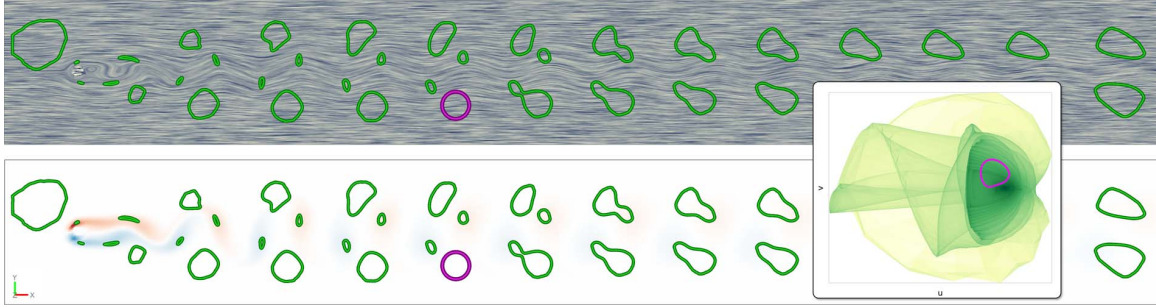


Figure 1: A 2D vector field of a Kármán vortex street, with a purple curve marking a part of the domain and the preimage of the curve's image in green. The inset shows a continuous scatterplot [1], with the curve's image in purple.

ABSTRACT

Extracting level sets from scalar data is a fundamental operation in visualization with many applications. Recently, the concept of level set extraction has been extended to bivariate scalar fields. Prior work on vector field equivalence, wherein an analyst marks a region in the domain and is shown other regions in the domain with similar vector values, pointed out the need to make this extraction operation fast, so that analysts can work interactively. To date, the fast extraction of level sets from bivariate scalar fields has not been researched as extensively as for the univariate case. In this paper, we present a novel algorithm that extracts fiber lines, i.e., the preimages of so called control polygons (FSCP), for bivariate 2D data by joint traversal of bounding volume hierarchies for both grid and FSCP elements. We performed an extensive evaluation, comparing our method to a two-dimensional adaptation of the method proposed by Klacansky et al., as well as to the naive approach for fiber line extraction. The evaluation incorporates a vast array of configurations in several datasets. We found that our method provides a speedup of several orders of magnitudes compared to the naive algorithm and requires two thirds of the computation time compared to Klacansky et al. adapted for 2D.

Keywords: Visualization, scalar fields, bivariate data, fibers, preimage extraction algorithm, dual bounding volume hierarchy traversal.

Index Terms: Human-centered computing—Visualization

1 INTRODUCTION & BACKGROUND

Level set methods such as isosurfaces and isolines are fundamental analytic tools in the study of scalar field data, because they often mark useful separations in the data. Much research effort in visualization was put into making their computation fast, as is evidenced in the Visualization Handbook [12], especially to allow interactive analysis. In many real-world domains such as mechanical engineering, fluid mechanics, and environmental sciences, it is beneficial to

investigate the multiple scalar quantities jointly. However, visual analysis of such multivariate data is generally challenging [11]. Carr et al. [5] presented fiber surfaces as a conceptual extension of isosurfaces to bivariate data, but to date there has only been limited research effort to make their extraction fast.

Formally, a *level set* of a function $f : D \rightarrow \mathbb{R}$, $D \subseteq \mathbb{R}^d$ to a value $v \in \mathbb{R}$ is the preimage of v under f : $f^{-1}(v) = \{x \in D \mid f(x) = v\}$. Analogously, a *fiber* of a bivariate function $f : \mathbb{R}^d \rightarrow \mathbb{R}^2$ to a combination of values $(u, v) \in \mathbb{R}^2$ is (u, v) 's preimage with respect to f : $f^{-1}(u, v) = \{x \in D \mid f(x) = (u, v)\}$, i.e., all points in the domain where f takes on a particular value combination. While level sets are usually $(d-1)$ -dimensional objects, e.g., surfaces in 3D, fibers of bivariate functions are typically $(d-2)$ -dimensional objects, e.g., lines in 3D. Carr et al. [5] introduced *fiber surface control polygons* (FSCP) P approximating a closed curve in the range of f (also called f 's *codomain*), defined *fiber surfaces* as the preimage of P under f : $f^{-1}(P) = \{x \in D \mid f(x) \in P\}$, and showed that this typically gives a surface in 3D again. For users to specify and manipulate the control polygon interactively, the fiber extraction has to be fast enough.

The algorithm by Carr et al. [5] used distance fields to a closed non-self-intersecting control polygon, but was not exact. Klacansky et al. [16] presented an algorithm for both fast and exact extraction of fiber surfaces for bivariate 3D scalar fields using piecewise-linear interpolation, i.e., the bivariate function is defined on a tetrahedral grid and each grid cell uses linear interpolation. For each edge e of the control polygon and each cell c of the grid, f 's preimage of e restricted to c can be computed exactly: (a) e is elongated to an infinite line l , (b) the signed distance to l is computed for each vertex of c , (c) marching tetrahedra is applied on c with the signed distance field and (d) the result is clipped to the original range of e . The fiber surface is then the union of the preimages for all combinations of m edges with n cells, giving a run-time in $O(m \cdot n)$.

The cell/edge preimage computation involves many arithmetic operations and tests, making extraction using this naive algorithm prohibitively slow for interactive applications. Additionally, only few cell/edge combinations have a nonempty preimage, i.e. contribute to the fiber surface. For these two reasons, Klacansky et al. [16]'s method computes a *bounding volume hierarchy* (BVH) using *axis-aligned bounding boxes* (AABBs) of the cell's images and queried it for each polygon edge using an adaptation of well known ray casting BVH traversal techniques. There is an overhead for the construction of the BVH, but this has to be performed only

*e-mail: raith@informatik.uni-leipzig.de

†e-mail: nsonga@informatik.uni-leipzig.de

‡e-mail: scheuer@informatik.uni-leipzig.de

§e-mail: heine@informatik.uni-leipzig.de

once if all interaction affects the control polygon only. Also, the result of the bounding volume hierarchy traversal can yield *false positives*, i.e., cell/edge combinations that do not intersect even though the bounding box of the cell intersects the edge. In practice, the algorithm's run-time grows nearly logarithmic with the number of cells, but it is still linear in the number of control polygon's edges. Klacansky et al. [16] considered control polygons only up to 11 edges and reported an average speed-up of roughly one order of magnitude compared to the naive algorithm. In many applications of fiber surfaces (see Fig. 1 and Sect. 2), the control polygons can comprise many more edges and raises the question for algorithms that scale sublinearly with the number control polygon edges.

Our work aims to further speed up the extraction of preimages for bivariate data when the number of control polygon edges is large. In this paper we focus on 2D domains, like Zheng et al. [36], who gave use cases for complex control polygons. We will refer to the extracted features as *fiber lines*. Central to our approach is the use of two BVHs, one for the cells and one for the control polygon edges, and traversing both BVHs jointly. This *Dual-BVH traversal (DBt)* common practice in computer graphics and robotics for collision detection and can be described as a recursive algorithm: Given a pair of tree nodes l, r from both trees, if their bounding volumes intersect, recurse into either l 's children or r 's children – usually based on the size of l 's and r 's bounding volume. The algorithm is initialized with the roots of both BVHs and reports all combinations of leaves whose bounding volumes intersect. For consistency, we use AABBs as bounding volumes for the edge BVH, too, even though, this will most likely produce more false positives compared to Klacansky et al. [16]. On the other hand, the dual traversal reduces the number of intersection tests, each of which use fewer arithmetic operations. However, it is not immediately clear whether the reduction in intersection test costs is worth the overhead of building the BVH for the control polygon and treatment of additional false positives. We find that a blend of Dual-BVH and Klacansky combines the best of both methods. The contributions of this paper are: (1) a novel algorithm for the extraction of fiber lines, combining the approach by Klacansky et al. with *DBt*, and (2) results from an extensive evaluation of the different methods, studying factors such as control polygon size and recursion strategies, in both theoretical and practical test settings.

2 RELATED WORK

An overview of visualization for multivariate data is given by Fuchs and Hauser [10]. Standard techniques for the depiction of multivariate data include glyphs [7]. In visual analysis it is typical to select important features interactively, but there are also automatic approaches, e.g. the work by Jänicke et al. [14]. The importance of feature-based techniques was highlighted in the work of Obermaier et al. [19] and Carr [4]. Sauber et al. [26] used a multifield graph for 3D scalar fields. Each node corresponds to a correlation field combining at least two fields, aiding the selection which correlation fields to investigate further. Nagaraj et al. [18] define a measure of scalar field similarity based on the field's gradients' similarity.

Fiber surfaces as an extension of isosurfaces to multifields were proposed by Carr et al. [5] based on earlier work by Saeki [23] on singular fibers. The method determines the preimage of a control polygon in the range of a bivariate function as the isosurface to the zero-level of a field coding the signed distance to the control polygon. It restricted the control polygon to be closed and non-self-intersecting, and only gave an approximate result, because the signed distance field was stored using piecewise interpolation. Wu et al. [33] presented a volume ray casting algorithm to display fiber surfaces. A different approach is taken by Jankowai and Hotz [15]. They use feature level sets with traits to define so-called standard isosurfaces, and fiber surfaces are special cases of these feature level sets. Klacansky et al. [16] presented an exact method to compute

fiber surfaces for bivariate functions defined on tetrahedral meshes using piecewise linear interpolation, and introduced using spatial search structures for objects in the function's range.

Concerning applications, Sakurai et al. [25] presented a tool to study singular fibers interactively, and in another work, Sakurai et al. [24] generalized the flexible isosurfaces interface [6] to fiber surfaces. Tierny and Carr [31] link Jacobi sets [8] and Reeb spaces [9] with fiber surfaces by using the Jacobi sets as seeds to determine fiber surface components that subdivide the domain into Reeb space components. Zheng et al. [36] study 2D vector field equivalence, which can be expressed as the preimage of the image of an object in the domain of the function. They noted that their method was very slow. Raith et al. [20,21] study trivariate functions arising from the study of 3D tensor field invariants, and its application and usefulness were shown in different settings [3,22]. Their approach has since been extended to general multivariate data by Blecha et al. [2]. Recently, Sharma et al. [28] used fiber surface control polygons seeded from continuous scatter plots to analyze electronic transitions. All these applications have in common that they usually require the extraction of preimages for control polygons and surfaces with many elements in typically interactive settings. In the work of Sharma and Natarajan [29], the Jacobi set is used to identify interesting barrel surface components. For this purpose, they present an output-sensitive approach for their calculation.

Our work is based on well-established results from computer graphics in the area of collision detection. A survey is given by [30].

3 METHOD

In this section, we describe our approach of combining *Dual-BVH traversal (DBt)* with Klacansky's approach. We also address important optimization issues such as the choice of recursion strategies, and explain the importance of the false positive ratio. In doing so, we show how this can be reduced without incurring a time loss in the corresponding preprocessing step. The algorithm here consists of two consecutive parts, i.e., a search space reduction using *DBt* (cf. Algorithm 1) followed by an extraction algorithm.

3.1 Extraction Algorithm

We begin by briefly elaborating on the base extraction algorithm, which is a standard implementation for extracting fiber lines adjusted for two-dimensional data. Given the field $f : D \rightarrow \mathbb{R}^2$, with $D \subseteq \mathbb{R}^2$ defined on a triangular mesh mapped to the range of f in which also the FSCP is defined. The algorithm now computes the coefficients for the transformation of the FSCP edges from the range to the domain D . For this purpose, the line coordinates are first normalized for each edge. If thereby an edge has a length of 0.0, it is skipped. Then we iterate over each triangular cell of the mesh for the current edge and calculate the signed distance between the edge and every vertex of the triangle. Using the signed distances, we apply marching triangles. We then calculate the intersection points of the FSCP edge with the triangle edges normalized to the values between 0 and 1. If both intersection points are on one side of the triangle, the triangle is skipped, otherwise, the line is truncated inside the triangle. Finally, we map the resulting intersection points to the domain D . A pseudocode implementation for the extraction algorithm can be found in the supplemental material.

3.2 Acceleration structure

As discussed earlier, Klacansky et al. [16] accelerate the algorithm by using a BVH. Instead of directly applying the fiber surface extraction, they first perform intersection tests between AABB and the FSCP edges. Our idea is to replace the intersection tests between lines and AABBs with intersection tests between pairs of AABBs. As the corresponding AABBs can be represented as two sets of coordinates spanning the AABBs, an intersection test between two AABBs amounts to a comparison of their coordinates. In addition to a

reduced computational cost, using a BVH for the FSCP promises an efficient search space reduction, also for complex FSCP with many edges. When intersecting using Klcansky's approach, a m number of FSCP edges and a BVH for the bivariate field in the range of f , each line must be compared to the BVH, resulting in a time-complexity linear in m and in theory logarithmic in n , where n is the number of cells. Utilizing two BVHs, the time-complexity can be reduced to logarithmic in m in theory. A downside is that an AABB does not approximate a line well and therefore tests between two AABBs may lead to more false positives than tests between an AABB and a line. To prevent this, once the BVH for the FSCP is in the leaves we use the test between an AABB and a line. This allows us to benefit from the low-cost calculation of the intersection between two AABBs, and at the same time from the more accurate tests between an AABB and a line. This provides an overall advantage for complex FSCP, making it more suitable for use in real-time applications.

We denote T as a BVH, which can either be constructed using the FSCP edges or using the triangles in the range as input. Our acceleration structure is then as follows: We first construct the two BVH T_1 and T_2 using the triangles and FSCP edges, respectively.

First, we test if the AABB in the roots T_1 and T_2 overlap at all. If this is not the case, the algorithm is aborted. Otherwise, we subdivide T_1 and check if it is child c and T_2 are leaves. If this is the case then a cell-line pair is formed and stored accordingly in the list R . If this is not the case, it is checked whether all children c intersect with T_2 using the intersection strategy. This intersection strategy is the most important part of the algorithm and decides if c or T_2 is a leaf node and at the same time contains the FSCP edges, that we use the tests between an AABB and a line. Otherwise, we use the test between two AABB for the intersection test. If they do not intersect, the corresponding c is skipped. Otherwise, the recursion strategy is used to check whether c and T_2 must be interchanged and the algorithm is called recursively accordingly. The algorithm returns the list R at the end, which contains all possible cell line pairs.

Algorithm 1 Speedup with BVH Algorithm

Require: BVH from dataset in codomain (triangles) T_1 , BVH from control polygon in codomain (lines) T_2 , return value R

```

1: set  $n_1 = \text{rootOf}(T_1)$ ,  $n_2 = \text{rootOf}(T_2)$ 
2: function TRAVERSE( $n_1$ ,  $n_2$ ,  $R$ ):
3:   for all children  $c$  of  $n_1$  do
4:     if isLeaf( $c$ ) and isLeaf( $n_2$ ) then
5:       add combination from cells in  $c$  &  $n_2$  to  $R$ 
6:     else if intersectionStrategy( $c$ ,  $n_2$ ) then
7:       if recursionStrategy then
8:         TRAVERSE( $c$ ,  $n_2$ ,  $R$ )
9:       else
10:        TRAVERSE( $n_2$ ,  $c$ ,  $R$ )
11:     end if
12:   end for
13: end function
```

3.3 Recursion Strategies

When searching through two BVHs, it is important to choose a reasonable strategy for selecting which BVH to reduce at every step to optimize search time, especially when BVH have different heights. A common approach is to select the tree with the greater height to reduce at each step, as this generally results in a faster overall search. However, it is also important to consider other factors such as the size and structure of the BVHs, as well as the search algorithm used, to determine the most effective reduction strategy. In addition, it may be beneficial to use heuristics or other techniques to guide the search and improve its efficiency. Ultimately, the choice of reduction strategy depends on the specific requirements and constraints of

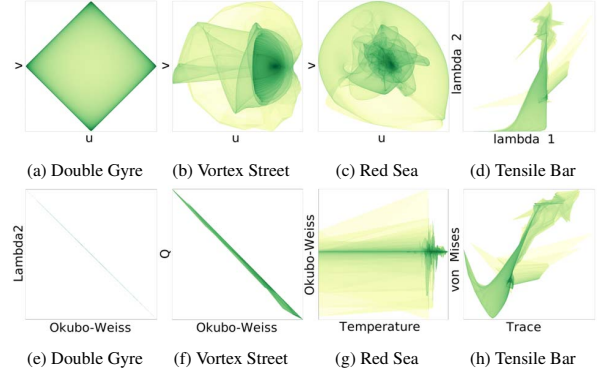


Figure 2: Continuous scatterplot for each dataset's range.

the search problem at hand. When investigating different swap strategies, we found the recursion strategy *area* to be most effective. This strategy is defined as the area of the current bounding volume processed by the algorithm. A detailed analysis of the investigated recursion strategies can be found in the supplemental material.

4 RESULTS

We demonstrate our algorithm and perform a comprehensive evaluation of three use cases. These are (I) the theoretical case, (II) the practical case, and (III) the field equivalence case. First, we describe the datasets and FSCP used. We then use the cases to compare our method with the comparison algorithms, the naive approach, DBt, and the Klcansky algorithm. For Klcansky, we used 8 elements per leaf, as this was reported as the best trade-off by Klcansky et al. [16]. For DBt and the blend between DBt and Klcansky we use 1 elements per leaf since we found this to give the best trade-off between search time and extraction time. To compare the search structures in isolation, we use the same extraction algorithm (see Sect. 3) for fiber lines with all search structures. All tests were performed on a workstation with an Intel Xeon E5-2630 v3 running at 2.40 GHz, 32 GB RAM, and an NVIDIA GeForce GTX 980. All algorithms are implemented using C/C++ and are not parallelized.

4.1 Datasets & Control Polygons

In this work, we use four well-known freely available datasets from the literature, whose continuous scatterplots [1] are represented in Fig. 2. One synthetic dataset, the Double Gyre [27] (64770 cells, Fig. 2a,e) and three real datasets, the Kármán Vortex Street from the Topology ToolKit [17, 32] data package (65536 cells, Fig. 2b,f), a common flow pattern, the Red Sea dataset from the 2020 SciVis contest [13, 34, 35] (498002 cells, Fig. 2c,g), and the Tensile Bar [37, 38] (9000 cells, Fig. 2d,h) from mechanics.

Furthermore, we used nine FSCP for the tests, for marking relevant regions in the codomain. The FSCPs range from 3 to 2997 edges. Note that Klcansky et al. only considered control polygons comprising up to 11 edges.

4.2 Evaluation in the Theoretical Case (I)

First, we compare in detail the total time and the quantities that actively affect it, and compare them with the comparison algorithms in case (I). We use all the datasets and intersect them with the prefabricated FSCP from Fig. 3. We position the center of the FSCP at 205 random positions in the range of AABB of the corresponding dataset codomains given in Fig. 2a,b,c,d. The Table 1 shows the evaluation for the total time across all 184,500 test cases. Mean values over all test cases are given for the values. It can be seen, that our algorithm is faster on average over all datasets compared to the other methods and has less search overhead. For example, our algorithm achieves a speedup of 2 orders of magnitude over the

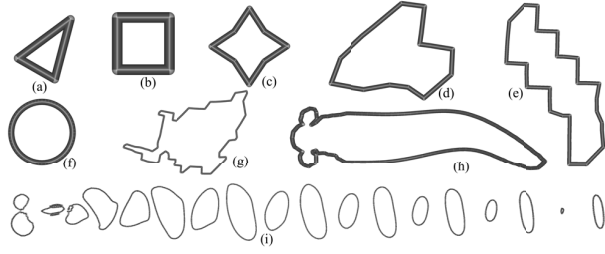


Figure 3: Visualization of the 9 control polygons in the codomain. Number of edges: (a)=3, (b)=4, (c)=8, (d)=16, (e)=38, (f)=60, (g)=126, (h)=232, and (i)=2997.

Table 1: Evaluation of case (I) with total time consisting of time for creation of additional BVH if available, intersection time, and extraction time in ms, number of intersection tests (NiT), and ratio true positives vs. all positives (TPaP). All values are averages, and bold are the best values.

Methode	Dataset	NiT	TPaP	Total Time
Naive approach	Double Gyre	–	0.11%	1048.36
	Kármán V. S.	–	0.15%	1059.19
	Red Sea	–	0.04%	7997.22
	Tensile Bar	–	0.25%	154.68
Klacansky BVH 8 elems. per leaf	Double Gyre	7160.60	50.04%	0.87
	Kármán V. S.	43856.45	17.95%	3.31
	Red Sea	45205.72	23.42%	3.96
	Tensile Bar	9271.62	24.97%	0.99
DBt 1 elem. per leaf	Double Gyre	16626.74	47.94%	1.99
	Kármán V. S.	35094.51	40.06%	3.92
	Red Sea	115691.46	42.68%	5.01
	Tensile Bar	17333.14	40.38%	1.22
Our Algorithm 1 elem. per leaf	Double Gyre	9363.42	75.06%	0.80
	Kármán V. S.	26937.60	54.45%	1.73
	Red Sea	40804.39	61.73%	2.29
	Tensile Bar	7765.37	51.08%	0.65

naive approach, requiring two-thirds of the total time of Klacansky, and is more than twice as fast as the DBt. The details also show that there is a significant difference between datasets. For example, the difference in the synthetic dataset is smaller than in the real datasets. This may be due to the codomain, which is very regular in the analytic dataset and contains hardly any overlapping cells, as can also be seen in the scatterplot of Fig. 2a.

4.3 Evaluation in the Practical Case (II)

During the automatic generation of FSCP, a high number of edges are frequently generated, which always lie completely in the codomain of the object. Therefore, individual edges cannot be excluded at an early stage. To investigate this for practical use cases, we consider the case (II) and compare the total time. Case (II) automatically generates a FSCP for the algorithm, in which an isoline is created for a given iso value and projected from the domain into the codomain. The dataset codomains are shown in Fig. 2e,f,g,h. We create 1001 FSCP per dataset by corresponding isovalues. This gives a total number of 4004 tests for case (II). Compared to the naive approach and Klacansky, our algorithm requires less total time (see Fig. 4). In particular, for the naive approach, we see that our algorithms are on average 1.67 orders of magnitude faster and in the best case even 1.76 orders of magnitude faster, which is even better than in case (I). Compared to Klacansky, our algorithm is 2.2 times faster. Only DBt performs significantly better than in Case (I) and is about as fast.

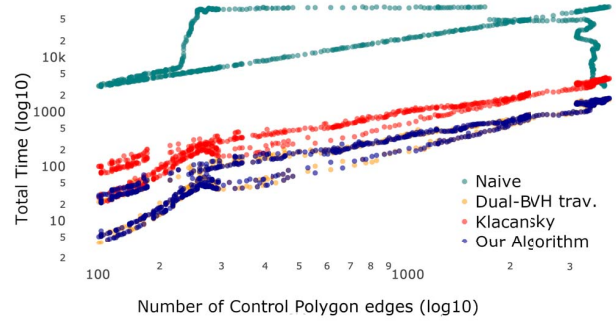


Figure 4: Plot the dataset Red Sea in the case (II).

4.4 Evaluation in the Field Equivalence Case (III)

Case (III) is borrowed from the work of Zheng et al. [36] in which field equivalence was calculated. However, as they admitted, their calculations were very slow. Our algorithm can reproduce the field equivalences by running two algorithms in sequence. We examine the overall time, i.e., the total time of the first and second algorithms, to determine whether we are also faster than the comparison algorithms. To mark the field equivalence, we use the FSCP from Fig. 3f and move it over the domain. Its fiber line is used as the new FSCP, to represent the field equivalence as a fiber line, as shown in Fig. 1. For our tests, we use the codomain as shown in Fig. 2e,f,g,h. For replication, we positioned the control polygon at 101 positions per dataset using the codomain from Fig. 2e,f,g,h, to compute field equivalences, resulting in 404 test cases. This results in a mean overall time of 40.23 ms, which allows for interactive analysis. When considering the Kármán vortex street, which Zheng et al. also considered, we require a mean overall time of 37.99 ms, indicating that our algorithm is faster than that of Zheng et al. [36]. We have found that our algorithm is on average 2.2 orders of magnitude faster than the naive approach, we are 1.75 times faster than Klacansky, and we are 1.19 times faster than the DBt.

5 CONCLUSION & FUTURE WORK

In this paper, we have presented a new algorithm that speeds up the extraction of fiber lines for bivariate 2D scalar fields. We have performed an extensive run-time comparison and shown that our algorithm is several orders of magnitude faster than the naive approach and also requires less total time than the adapted algorithm of Klacansky and DBt. To this end, we compared the algorithms in three cases. We optimized and evaluated individual components of the algorithm in case (I) and compared their final results with naive, DBt, and Klacansky. We found, that our approach is particularly well suited for FSCP with many edges. In case (II), we investigated the relationship between the total time and the number of edges. In case (III), we used our algorithm to extract field equivalents and similarly this is faster than the naive algorithm and Klacansky.

Our research has shown that the total time is mostly dominated by the extraction time. Further improvements are thus to be expected from reducing false positive cell/edge combinations. One could also extend the framework by Zheng et al. [36] to use our method. The results of our work suggest that it would be significantly faster. Moreover, our results indicate a higher benefit of using our approach over a 2D version of Klacansky et al. [16] when the number of edges in the FSCP is large. In future work, we will study the performance of our algorithm adapted to 3D domains, because most applications focus on this case.

ACKNOWLEDGMENTS

This work was funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) - SCHE 663/17-1.

REFERENCES

- [1] S. Bachthaler and D. Weiskopf. Continuous scatterplots. *IEEE Transactions on Visualization and Computer Graphics*, 14(6):1428–1435, 2008.
- [2] C. Blecha, F. Raith, A. J. Präger, T. Nagel, O. Kolditz, J. Maßmann, N. Röber, M. Böttinger, and G. Scheuermann. Fiber surfaces for many variables. *Computer Graphics Forum*, 39(3):317–329, 2020.
- [3] C. Blecha, F. Raith, G. Scheuermann, T. Nagel, O. Kolditz, and J. Maßmann. Analysis of coupled thermo-hydro-mechanical simulations of a generic nuclear waste repository in clay rock using fiber surfaces. In *2019 IEEE Pacific Visualization Symposium (PacificVis)*, pp. 189–201, 2019.
- [4] H. Carr. Feature analysis in multifields. In Hansen et al. [11], pp. 197–204.
- [5] H. Carr, Z. Geng, J. Tierny, A. Chattopadhyay, and A. Knoll. Fiber surfaces: Generalizing isosurfaces to bivariate data. *Computer Graphics Forum*, 34(3):241–250, 2015.
- [6] H. Carr, J. Snoeyink, and M. van de Panne. Flexible isosurfaces: Simplifying and displaying scalar topology using the contour tree. *Computational Geometry*, 43(1):42–58, 2010. Special Issue on the 14th Annual Fall Workshop.
- [7] D. H. Chung, R. S. Laramée, J. Kehrner, and H. Hauser. Glyph-based multi-field visualization. In *Scientific Visualization*, pp. 129–137. Springer, 2014.
- [8] H. Edelsbrunner and J. Harer. Jacobi sets of multiple Morse functions. In F. Cucker, R. DeVore, P. Olver, and E. Süli, eds., *Foundations of Computational Mathematics*, Minneapolis 2002, number 312 in London Mathematical Society Lecture Note Series, pp. 37–57. Cambridge University Press, 2004.
- [9] H. Edelsbrunner, J. Harer, and A. K. Patel. Reeb spaces of piecewise linear mappings. In *Proceedings of the Twenty-Fourth Annual Symposium on Computational Geometry*, SCG ’08, p. 242–250. Association for Computing Machinery, New York, NY, USA, 2008.
- [10] R. Fuchs and H. Hauser. Visualization of multi-variate scientific data. *Computer Graphics Forum*, 28(6):1670–1690, 2009.
- [11] C. Hansen, M. Chen, C. Johnson, A. Kaufman, and H. Hagen, eds. *Scientific Visualization: Uncertainty, Multifield, Biomedical, and Scalable Visualization*. Mathematics and Visualization. Springer, 2014.
- [12] C. D. Hansen and C. R. Johnson, eds. *Visualization Handbook*. Butterworth-Heinemann, Burlington, 2005.
- [13] I. Hoteit, X. Luo, M. Bocquet, A. Kohl, and B. Ait-El-Fquih. Data assimilation in oceanography: current status and new directions. *New frontiers in operational oceanography*, pp. 465–512, 2018.
- [14] H. Janicke, A. Wiebel, G. Scheuermann, and W. Kollmann. Multifield visualization using local statistical complexity. *IEEE Transactions on Visualization and Computer Graphics*, 13(6):1384–1391, 2007.
- [15] J. Jankowai and I. Hotz. Feature level-sets: Generalizing iso-surfaces to multi-variate data. *IEEE Transactions on Visualization and Computer Graphics*, 26(2):1308–1319, 2020.
- [16] P. Klacansky, J. Tierny, H. Carr, and Z. Geng. Fast and exact fiber surfaces for tetrahedral meshes. *IEEE Transactions on Visualization and Computer Graphics*, 23(7):1782–1795, 2017.
- [17] T. B. Masood, J. Budin, M. Falk, G. Favelier, C. Garth, C. Gueunet, P. Guillou, L. Hofmann, P. Hristov, A. Kamakshidasan, C. Kappe, P. Klacansky, P. Laurin, J. A. Levine, J. Lukasczyk, D. Sakurai, M. Soler, P. Steneteg, J. Tierny, W. Usher, J. Vidal, and M. Wozniak. An overview of the topology toolkit. In I. Hotz, T. Bin Masood, F. Sadlo, and J. Tierny, eds., *Topological Methods in Data Analysis and Visualization VI*, pp. 327–342. Springer International Publishing, Cham, 2021.
- [18] S. Nagaraj, V. Natarajan, and R. S. Nanjundiah. A gradient-based comparison measure for visual analysis of multifield data. *Computer Graphics Forum*, 30(3):1101–1110, 2011.
- [19] H. Obermaier and R. Peikert. Feature-based visualization of multifields. In Hansen et al. [11], pp. 189–196.
- [20] F. Raith, C. Blecha, T. Nagel, F. Parisio, O. Kolditz, F. Günther, M. Stommel, and G. Scheuermann. Tensor field visualization using fiber surfaces of invariant space. *IEEE Transactions on Visualization and Computer Graphics*, 25(1):1122–1131, 2019.
- [21] F. Raith, C. Blecha, K. Rink, W. Wang, O. Kolditz, H. Shao, and G. Scheuermann. Visual analysis of a full-scale-emplacement experiment in the underground rock laboratory Mont Terri using fiber surfaces. In *Proceedings of the Workshop on Visualisation in Environmental Sciences*. The Eurographics Association, 2020.
- [22] F. Raith, G. Scheuermann, and C. Gillmann. Uncertainty-aware detection and visualization of ocean eddies in ensemble flow fields—a case study of the red sea. In *Proceedings of the Workshop on Visualisation in Environmental Sciences*, 2021.
- [23] O. Saeki. *Topology of Singular Fibers of Differentiable Maps*. Number 1854 in Lecture Notes in Mathematics. Springer-Verlag, 2004.
- [24] D. Sakurai, K. Ono, H. Carr, J. Nonaka, and T. Kawanabe. Flexible fiber surfaces: A Reeb-free approach. In H. Carr, I. Fujishiro, F. Sadlo, and S. Takahashi, eds., *Topological Methods in Data Analysis and Visualization V*. Springer International Publishing, Cham, 2020.
- [25] D. Sakurai, O. Saeki, H. Carr, H.-Y. Wu, T. Yamamoto, D. Duke, and S. Takahashi. Interactive visualization for singular fibers of functions $f: R^3 \rightarrow R^2$. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):945–954, 2016.
- [26] N. Sauber, H. Theisel, and H. Seidel. Multifield-graphs: An approach to visualizing correlations in multifield scalar data. *IEEE Transactions on Visualization and Computer Graphics*, 12(5):917–924, 2006.
- [27] S. C. Shadden, F. Lekien, and J. E. Marsden. Definition and properties of Lagrangian coherent structures from finite-time Lyapunov exponents in two-dimensional aperiodic flows. *Physica D: Nonlinear Phenomena*, 212(3–4):271–304, 2005.
- [28] M. Sharma, T. B. Masood, S. S. Thygesen, M. Linares, I. Hotz, and V. Natarajan. Continuous scatterplot operators for bivariate analysis and study of electronic transitions. *IEEE Transactions on Visualization and Computer Graphics*, 2023. to appear.
- [29] M. Sharma and V. Natarajan. Jacobi set driven search for flexible fiber surface extraction. In *2022 Topological Data Analysis and Visualization (TopoInVis)*, pp. 49–58, 2022.
- [30] H. A. Sulaiman and A. Bade. Bounding volume hierarchies for collision detection. *Computer Graphics*, 10:5772–2386, 2012.
- [31] J. Tierny and H. Carr. Jacobi fiber surfaces for bivariate Reeb space computation. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):960–969, 2017.
- [32] J. Tierny, G. Favelier, J. A. Levine, C. Gueunet, and M. Michaux. The Topology ToolKit. *IEEE Transactions on Visualization and Computer Graphics (Proc. of IEEE VIS)*, 2017. <https://topology-tool-kit.github.io/>.
- [33] K. Wu, A. Knoll, B. J. Isaac, H. Carr, and V. Pascucci. Direct multifield volume ray casting of fiber surfaces. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):941–949, Jan 2017.
- [34] P. Zhan, G. Krokos, D. Guo, and I. Hoteit. Three-dimensional signature of the red sea eddies and eddy-induced transport. *Geophysical Research Letters*, 46(4):2167–2177, 2019.
- [35] P. Zhan, A. C. Subramanian, F. Yao, and I. Hoteit. Eddies in the red sea: A statistical and dynamical study. *Journal of Geophysical Research: Oceans*, 119(6):3909–3925, 2014.
- [36] B. Zheng, B. Rieck, H. Leitte, and F. Sadlo. Visualization of equivalence in 2D bivariate fields. *Computer Graphics Forum*, 38(3):311–323, 2019.
- [37] V. Zobel and G. Scheuermann. Extremal curves and surfaces in symmetric tensor fields. *The Visual Computer*, 34(10):1427–1442, 2018.
- [38] V. Zobel, M. Stommel, and G. Scheuermann. Visualizing gradients of stress tensor fields. In *Modeling, Analysis, and Visualization of Anisotropy*, pp. 65–81. Springer, 2017.